TRYHACKME · WRITEUP

Room 404

Divulgación de código fuente vía directorio **.git** expuesto

Raul Velasquez
Analista de Ciberseguridad · SOC / GRC · @rauljvc8
Octubre 2026 · Dificultad: Fácil · Categoria: Web / OSINT

404

1. Resumen ejecutivo

Room 404 es un reto de enumeracion web. Una aplicacion Flask corre en un puerto no estandar (8080) y expone publicamente su directorio de control de versiones **.git**. Al reconstruir ese repositorio es posible recuperar todo el codigo fuente de la aplicacion y descubrir una *staging flag* que quedo hardcodeada dentro de un archivo README.md que nunca debio llegar a produccion.

El hallazgo corresponde a un caso clasico de **Exposicion de Datos Sensibles** (OWASP A02:2021) combinado con exposicion de metadatos de control de versiones — una de las malas configuraciones web mas comunes y de mayor impacto.

Servicio objetivo	HTTP (Werkzeug/Flask 3.0.1) en el puerto 8080
Vulnerabilidad	Directorio .git/ accesible publicamente por HTTP
Impacto	Divulgacion total del codigo fuente + flag hardcodeada
Severidad	Alta
Herramientas	nmap, git-dumper, grep

Objetivos de la maquina

#	Objetivo	Estado
1	Dumpear el codigo fuente expuesto	Completado
2	Encontrar la flag	Completado

2. Reconocimiento — Nmap

El ejercicio inicia con un escaneo de servicios y versiones contra el objetivo, incluyendo los scripts por defecto de Nmap (`-sC`).

```
nmap -sV -sC 10.67.149.109
```

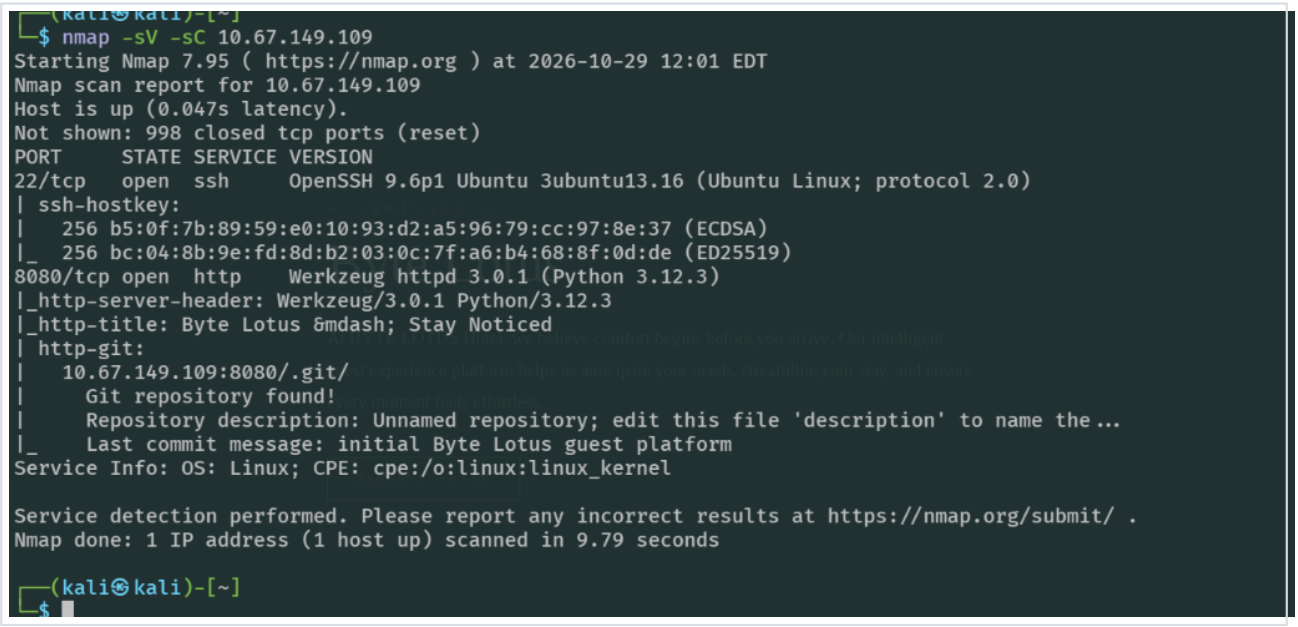


Figura 1 — Resultado del escaneo Nmap: puertos 22 y 8080 abiertos.

Puertos y servicios detectados

Puerto	Servicio	Version
22/tcp	SSH	OpenSSH 9.6p1 (Ubuntu)
8080/tcp	HTTP	Werkzeug httpd 3.0.1 (Python 3.12.3)

El dato decisivo lo aporta el script `http-git`, que detecta directamente el repositorio expuesto e incluso muestra el ultimo mensaje de commit:

```
| http-git:
|   10.67.149.109:8080/.git/
|     Git repository found!
|     Last commit message: initial Byte Lotus guest platform
```

3. Dump del codigo fuente expuesto

Dado que la carpeta **.git/** es accesible por HTTP, se reconstruye el repositorio completo con la herramienta `git-dumper`.

```
# Instalacion
pipx install git-dumper

# Dump (el argumento DIR es obligatorio)
git-dumper http://10.67.149.109:8080/.git ./room404.src
```

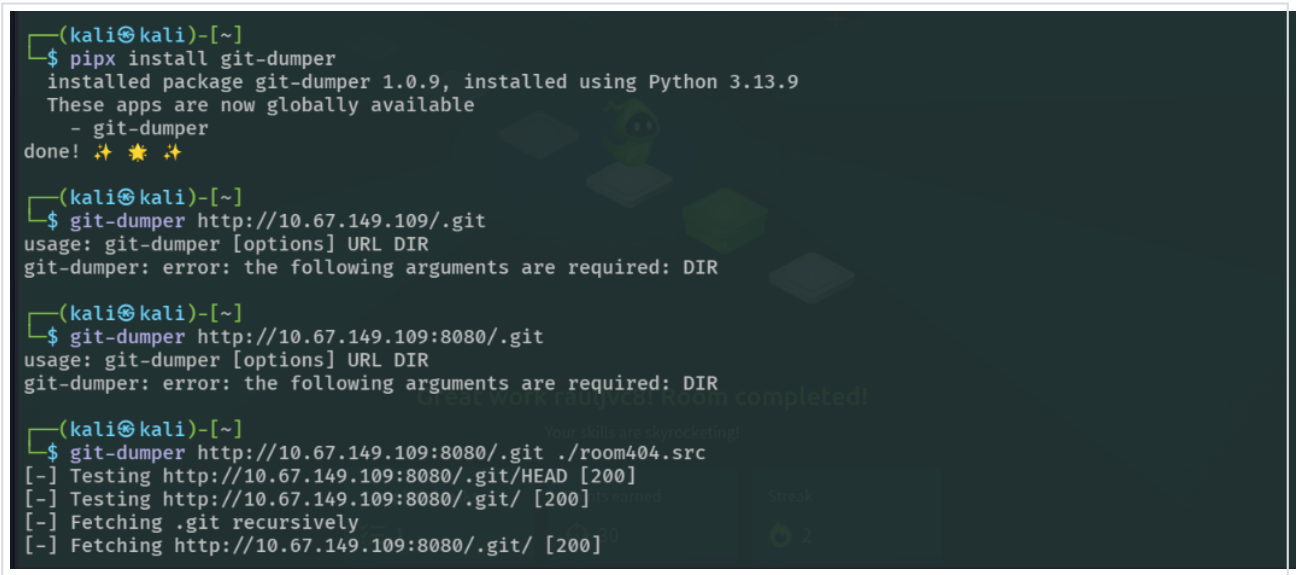


Figura 2 — git-dumper reconstruyendo el repositorio desde /.git/.

> **Leccion aprendida:** git-dumper requiere una URL **y** un directorio de salida. Ejecutarlo solo con la URL devuelve el error `the following arguments are required: DIR`.

4. Revision del repositorio recuperado

```
cd room404.src
ls -la
```

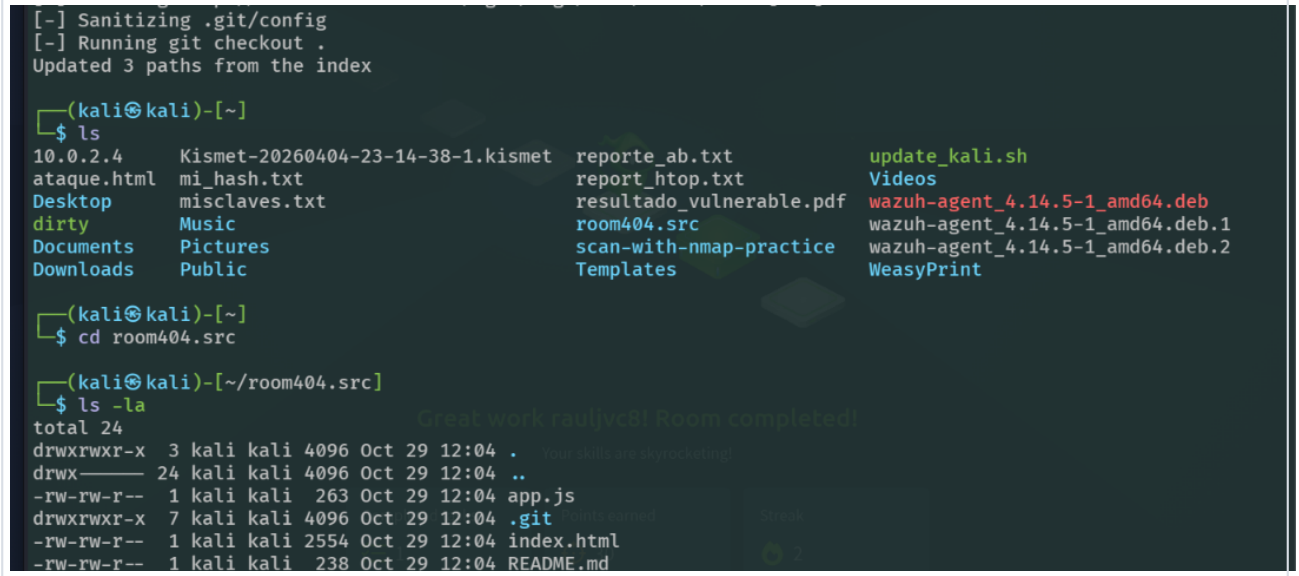


Figura 3 — Archivos recuperados del repositorio dumpeado.

Archivo	Proposito
app.js	Script del front-end
index.html	Pagina principal del sitio
README.md	Notas internas de staging

Archivo	Proposito
.git/	Historial completo de control de versiones

5. Obtencion de la flag

El archivo **README.md** dentro del repositorio es un documento interno de staging que explicitamente pedia no desplegarse en produccion — y aun asi se publico junto con el .git:

```
cat README.md
```

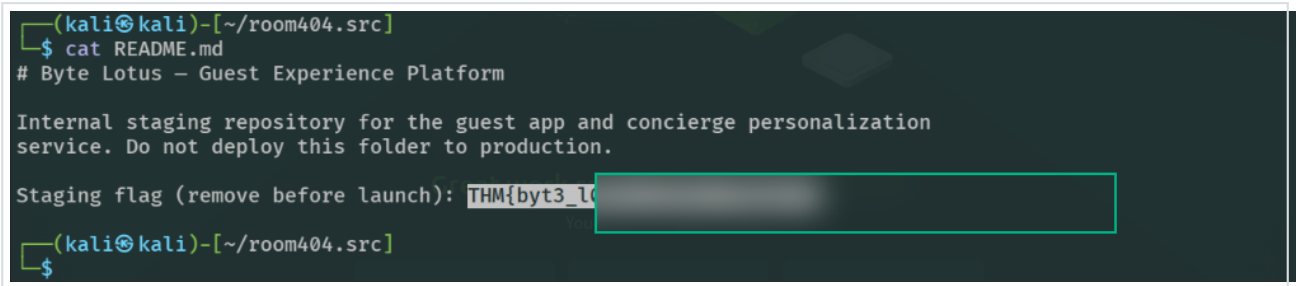


Figura 4 — Staging flag hardcodeada dentro del README.md.

FLAG CAPTURADA

THM{byt3_l0tus_...}

Nota: la flag se muestra parcialmente ocultada de forma intencional, al ser este un documento de portafolio de caracter publico.

Consejo para rooms similares: conviene siempre hacer grep de todo el dump buscando secretos, ya que la flag puede estar en commits antiguos y no en los archivos actuales:

```
grep -rniE "flag|thm\{|secret|password|token" .
git log -p --all | grep -iE "flag|thm\{|secret"
```

6. Remediacion

#	Recomendacion
1	Nunca exponer .git/ por HTTP. Bloquearlo en el servidor web (ej. 'location ~ /\.git { deny all; }' en Nginx).
2	Mantener secretos fuera del control de versiones: usar variables de entorno o un gestor de secretos.
3	Separar staging de produccion: notas y repos internos no deben desplegarse en hosts publicos.
4	Escanear el propio perimetro con herramientas como 'nmap --script http-git' antes que un atacante.
5	Purgar secretos filtrados del historial: borrar en un commit nuevo no basta, hay que rotar y reescribir.

Aprendizajes clave

- Un directorio .git/ accesible publicamente filtra toda la base de codigo y es de las fallas de mayor impacto.
- Los scripts por defecto de Nmap (-sC) muchas veces descubren la vulnerabilidad automaticamente.
- La divulgacion de codigo fuente suele derivar en credenciales hardcodeadas, API keys y endpoints internos.

Referencias: OWASP Top 10 (2021) A02 · git-dumper (github.com/arthaud/git-dumper) · Nmap http-git NSE script (nmap.org/nsedoc/scripts/http-git.html)
Informe elaborado por Raul Velasquez — Portafolio de Ciberseguridad SOC / GRC.